With examples in C#

ST

Street Coder

The rules to break and how to break them

Sedat Kapanoğlu



MANNING

Kapanoğlu



Street Coder

510.78PG K171

Street Coder

The rules to break and how to break them

Sedat Kapanoğlu

MANNING

With examples in C#

ST



1 To the streets

This chapter covers

- The realities of the streets
- Who is a street coder?
- The problems of modern software development
- How to solve your problems with street lore

I am lucky. I wrote my first program in the 1980s. It only required me to turn on the computer, which took less than a second, write 2 lines of code, type `RUN`, and voila! The screen was suddenly filled with my name. I was immediately awestruck by the possibilities. If I could do this with 2 lines, imagine what I could do with 6 lines, or even 20 lines! My nine-year-old brain was flooded with so much dopamine that I was addicted to programming at that instant.

Today, software development is immensely more complex. It's nowhere close to the simplicity of the 1980s, when user interactions only consisted of "press any key to continue," although users occasionally struggled to find an "any" key on their keyboard. There were no windows, no mice, no web pages, no UI elements, no libraries, no frameworks, no runtimes, no mobile devices. All you had was a set of commands and a static hardware configuration.

There is a reason for every level of abstraction we now have, and it's not that we are masochists, with the exception of Haskell[1] programmers. Those abstractions are in place because they're the only way to catch up with current software standards. Programming isn't about filling the screen with your name anymore. Your name must be in the correct font, and it must be in a window so you can drag it around and resize it. Your program must look good. It should support copy and paste. It must support different names for configurability as well. Perhaps it should store the names in a database, even in the cloud. Filling the screen with your name isn't so much fun anymore.

Fortunately, we have resources to contend with the complexity: universities, hackathons, boot camps, online courses, and *rubber ducks*.

> **TIP** Rubber duck debugging is an esoteric method for finding solutions to programming problems. It involves talking to a yellow plastic bird. I'll tell you more about it in the debugging chapter.

We should be well equipped with all the resources we have, but the foundation we build for ourselves may not always be sufficient in a high-competition, high-demanding career of software development, *the streets*.

1.1 What matters in the streets

The world of professional software development is quite mysterious. Some customers swear that they will pay you in a couple of days every time you call them for months on end. Some employers don't pay you any salary at all, but they insist they will pay you "once they make money." The chaotic randomness of the universe decides who gets the window office. Some bugs disappear when you use a debugger. Some teams don't use any source control at all. Yes, it's frightening. But you must face the realities.

One thing is clear in the streets: your throughput is what matters most. Nobody cares about your elegant design, your knowledge of algorithms, or your high-quality code. All they care about is how much you can deliver in a given time. Counterintuitively, good design, good use of algorithms, and good quality code can impact your throughput significantly, and that's what many programmers miss. Such matters are usually thought of as hindrances, frictions between a programmer and the deadline. That kind of thinking can turn you into a zombie with a ball and chain attached to your foot.

In fact, some people do care about the quality of your code: your colleagues. They don't want to babysit your code. They want your code to work and be easily understandable and maintainable. This is something you owe them because once you commit your code to the repository, it's everybody's code. In a team, the team's throughput is more important than that of each member. If you are writing bad code, you are slowing your colleagues down. Your code's lack of quality hurts the team, and a slowed-down team hurts the product, and an unreleased product hurts your career.

---

[1] Haskell is an esoteric language that was created as a challenge to fit as many academic papers as possible into a single programming language.

The easiest thing you can write from scratch is the idea, and the next easiest thing is the design. That's why good design matters. Good design isn't something that looks good on paper. You can have a design in your mind that works. You will encounter people who don't believe in designing and just improvise the code. Those people don't value their time.

Similarly, a good design pattern or a good algorithm can increase your throughput. If it doesn't help your throughput, it's not useful. Because almost everything can be given a monetary value, everything you do can be measured in terms of throughput.

You can have high throughput with bad code too, but only in the first iteration. The moment the customer requests a change, you are stuck with maintaining terrible code. Throughout this book, I'll be talking about cases in which you can realize that you are digging yourself into a hole and get yourself out of it before you lose your mind.

1.2 Who's a street coder?

Microsoft considers two distinct categories of candidates when hiring: new graduates of a computer science department and industry experts who have substantial experience in software development.

Be it a self-taught programmer or someone who studied computer science, they are missing a common piece at the beginning of their career: *street lore*, which is the expertise to know what matters most. A self-taught programmer has many trials and errors under their belt but can lack knowledge of formal theory and how it applies to everyday programming. A university graduate, on the other hand, knows a lot about theory but lacks practicality and, sometimes, a questioning attitude toward what they learned. See figure 1.1.

The corpus you learn at school doesn't have priority associated with it. You learn in the order of the learning path, not in the order of importance. You have no idea how useful certain subjects might be in the streets, where competition is relentless. Timelines are unrealistic. Coffee is cold. The best framework in the world has that single

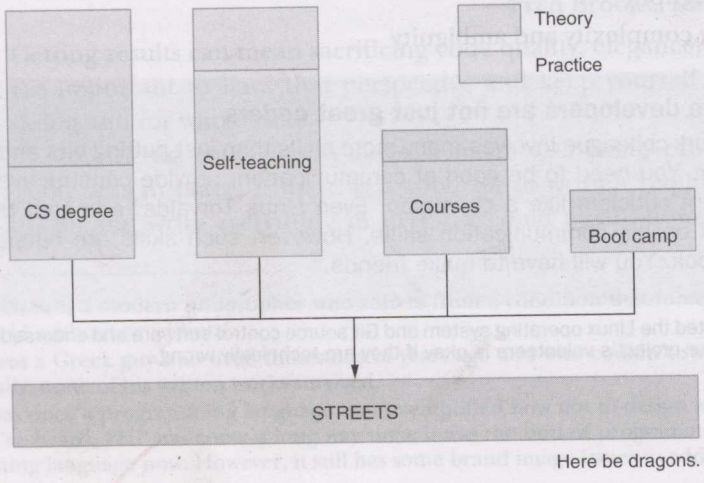


Figure 1.1 Starting a career through different paths

bug that renders a week of your work worthless. Your perfectly designed abstraction crumbles under pressure from the customer who constantly changes their requirements. You manage to quickly refactor your code with some copy-paste, but now you must edit 15 separate places just to change one configuration value.

Over the years, you develop new skills to tackle ambiguity and complexity. Self-taught programmers learn some algorithms that help them, and university graduates eventually understand that the best theory isn't always the most practical.

A street coder is anyone with software development experience in the industry who has had their beliefs and theories shaped by the realities of an unreasonable boss who wanted a week's worth of work done in the morning. They have learned to back up everything on multiple media after they lose thousands of lines of code and have to rewrite it all from scratch. They have seen C-beams glitter in the server room from burning hard drives and have fought with the systems administrator at the doors of the server room just to get access to production because someone has just deployed an untested piece of code. They have tested their software-compression code on its own source code, only to discover that it's compressed everything into one byte and the value of that byte is 255. The decompression algorithm is yet to be invented.

You've just graduated and are looking for a job, or you've been fascinated by programming but have no idea what awaits you. You've gotten out that boot camp and are looking for job opportunities, but you're not sure about the knowledge gap you have. You've taught yourself a programming language, but you're not sure what is missing from your skills tool belt. Welcome to the streets.

1.3 Great street coders

In addition to street cred, honor, and loyalty, a street coder ideally possesses these qualities:

- Questioning
- Results-driven (aka, "results-oriented" in HR-speak)
- High-throughput
- Embracing complexity and ambiguity

Great software developers are not just great coders

Being a great work colleague involves many more skills than just putting bits and bytes into a computer. You need to be good at communication, provide constructive feedback, and accept criticism like a champion. Even Linus Torvalds[a] admitted that he needed to work on his communication skills. However, such skills are outside the scope of this book. You will have to make friends.

[a] Linus Torvalds created the Linux operating system and Git source control software and endorsed the belief that swearing at your project's volunteers is okay if they are technically wrong.

1.3.1 Questioning

Someone talking to themselves is considered unusual at best, especially if they don't have answers to the questions they are asking themselves. However, being a questioning person, asking questions of yourself, asking questions about the most accepted notions, and deconstructing them can clarify your vision.

Many books, software experts, and Slavoj Žižek[2] emphasize the importance of being critical and inquisitive, but few of them give you something to work with. In this book, you'll find examples of very well-known techniques and best practices and how they can be less efficient than they claim to be.

A critique of a technique doesn't mean it's useless. However, it will expand your horizon so you can identify some use cases in which an alternative technique might actually be better.

The goal of this book isn't to cover every programming technique from end to end, but to give you a perspective on how to treat best practices, how to prioritize them based on merit, and how you can weigh the pros and cons of alternative approaches.

1.3.2 Results-driven

You can be the best programmer in the world with the best understanding of the intricacies of software development and can come up with the best design for your own code, but those will mean nothing if you are not shipping, if you are not getting the product out.

According to Zeno's paradox,[3] to reach an end goal, you must first reach the halfway-through point. It's a paradox because to reach the halfway-through point, you have to reach the quarter-way-through point, and so on, which makes you unable to reach anywhere. Zeno had a point: having an end product requires you to meet deadlines and the milestones in between, too. Otherwise, it's impossible to reach your end goal. Being results-driven also means being milestones-driven, being progress-driven.

> *"How does a project get to be a year late? ... One day at a time."*
>
> —Fred Brooks, *The Mythical Man Month*

Getting results can mean sacrificing code quality, elegance, and technical excellence. It's important to have that perspective and keep yourself in check for what you're doing and for whose sake.

Sacrificing code quality doesn't mean sacrificing product quality. If you have good tests in place, if there is a good set of written requirements, you can even go ahead and write everything in PHP.[4] It could mean, however, you will have to bear

---

[2] Slavoj Žižek is a modern philosopher who suffers from a condition that forces him to criticize everything in the world, without exceptions.

[3] Zeno was a Greek guy who lived thousands of years ago and who couldn't stop asking frustrating questions. Naturally, none of his written works survived.

[4] PHP was once a programming language that exemplified how not to design a programming language. From what I've heard, PHP has come a long way since it was the butt of programming jokes and is a fantastic programming language now. However, it still has some brand image issues to address.

some pain in the future because bad-quality code will eventually bite you. It's called code karma.

Some of the techniques you'll learn in the book will help you make decisions to get results.

1.3.3 High-throughput

The greatest factors affecting the speed of your development are experience, good and clear specifications, and mechanical keyboards. Just kidding—contrary to popular belief, mechanical keyboards don't help your speed at all. They just look cool and are great at annoying your significant other. In fact, I don't think typing speed helps development speed at all. Your confidence in your typing speed might even encourage you to write more elaborate code than is necessary.

Some expertise can be gained by learning from others' mistakes and despair. In this book, you'll find examples of such cases. The techniques and knowledge you gained will make you write less code and make decisions faster and will allow you to have as little technical debt as possible so you won't be spending days untangling code you wrote only six months ago.

1.3.4 Embracing complexity and ambiguity

Complexity is scary, and ambiguity more so, because you don't even know how much you should be scared and that scares you even more.

Dealing with ambiguity is one of the core skills Microsoft recruiters ask questions about in interviews. That usually entails hypothetical questions like "How many violin repair shops are there in New York?," "How many gas stations are there in Los Angeles?," or "How many Secret Service agents does the president have, and what's their shift schedule? List their names, and preferably show their walking paths on this blueprint of the White House."

The trick to solving these questions comes down to clarifying everything you know about the problem and arriving at an approximation based on those facts. For example, you can start with New York's population and how many people might be playing violin in the population. That would give you an idea about the size of the market and how much competition the market can support.

Similarly, when faced with a problem with some unknown parameters, such as estimating the time it would take to develop a feature, you can always narrow the window of approximation based on what you know. You can use what you know to your advantage and leverage it as much as possible, which can reduce the ambiguous part to minuscule.

Interestingly, dealing with complexity is similar. Something that looks extremely complex can be divided into parts that are much more manageable, less complex, and, in the end, simpler.

LOS ANGELES PUBLIC LIBRARY
3 7244 2433 6593 0

SOFTWARE DEVELOPMENT

Street Coder

Sedat Kapanoğlu

Fresh-faced CS grads, bootcampers, and other junior developers lack a vital quality: the "street smarts" of experience. To succeed in software, you need the skills and discipline to put theory into action. You also need to know when to go rogue and break the unbreakable rules. This book is your survival guide.

Street Coder teaches you how to handle the realities of day-to-day coding as a software developer. Self-taught guru Sedat Kapanoğlu shares down-and-dirty advice that's rooted in his personal hands-on experience, not abstract theory or ivory-tower ideology. You'll learn how to adapt what you've learned from books and classes to the challenges you'll face on the job. As you go, you'll get tips on everything from technical implementations to handling a paranoid manager.

What's Inside

- Beginner-friendly insights on code optimization, parallelization, and refactoring
- Put "bad" practices to good use
- Learn to love testing
- Embrace code breaks and become friends with failure

For new programmers. Examples in C#.

Sedat Kapanoğlu is a self-taught programmer with more than 25 years of experience, including a stint at Microsoft.

Register this print book to get free access to all ebook formats.
Visit https://www.manning.com/freebook

"Th
I wish I'd had when I started
in the world of application development."
—Janek López, Fosh Tech

"A must-read for every developer level!"
—Vincent Delcoigne, Wavenet

"Full of solid real-world programming advice with a touch of humor."
—Samuel Bosch, ILVO

"I started my career as a street coder, and wish I had read this book at that time so I could have avoided many detours and saved a few years of my career."
—Xu Yang, Ansteel Group

Free eBook
See first page

ISBN: 978-1-61729-837-0



MANNING $49.99 / Can $65.99 [INCLUDING eBOOK]

ST
Full Bmd
Bmc
#30

National Aeronautics and
Space Administration

Human Resources and
Office of Education

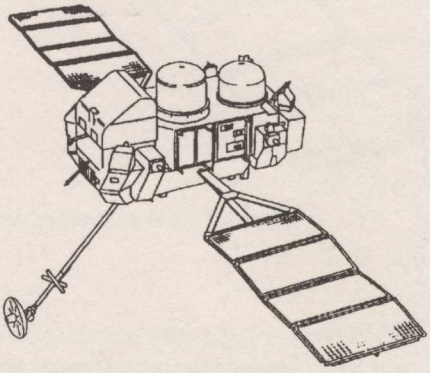
Education Division

Educational Product	
Teachers & Students	Grades 5 and up

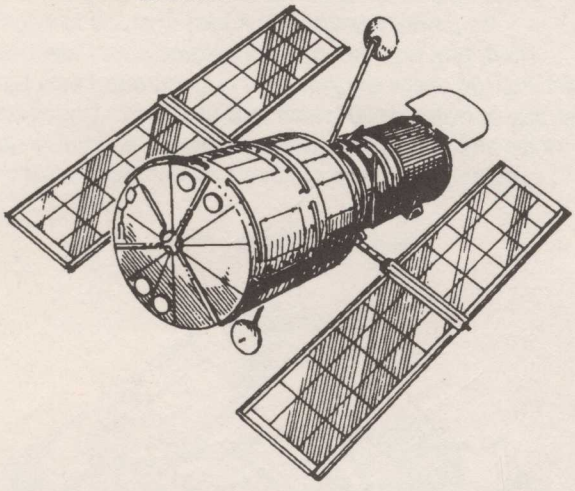
NASA's Great Observatories
Paper Model Kits

LOS ANGELES PU
CEN
DEPT. OF SCIE
630 W
LOS ANGEL

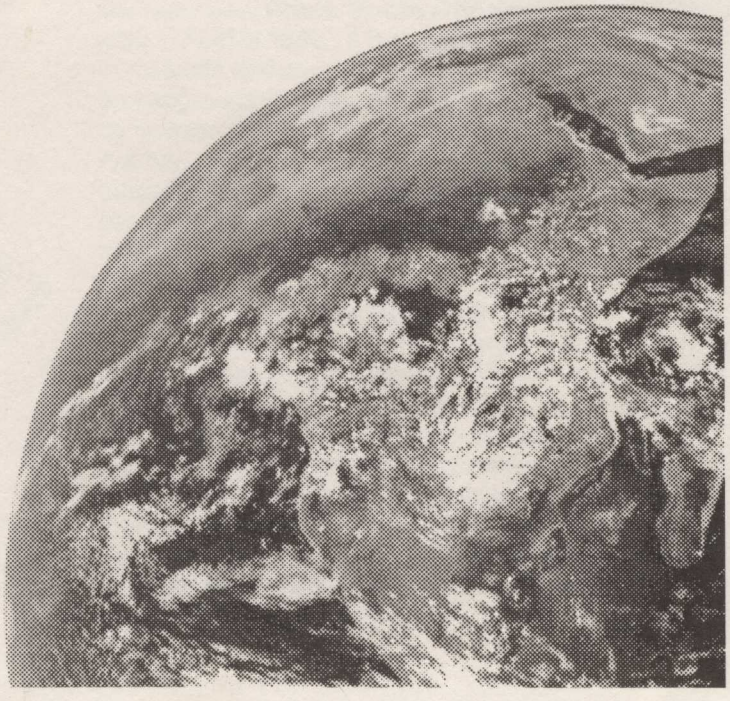
LOS ANGELES PUBLIC LIBRARY
CENTRAL LIBRARY
DEPT. OF SCIENCE, TECHNOLOGY & PATENTS
630 WEST 5th ST.
LOS ANGELES, CA. 90071



Compton Gamma Ray Observatory



Hubble Space Telescope



R 522.2 N2435

PED-136, August 1993

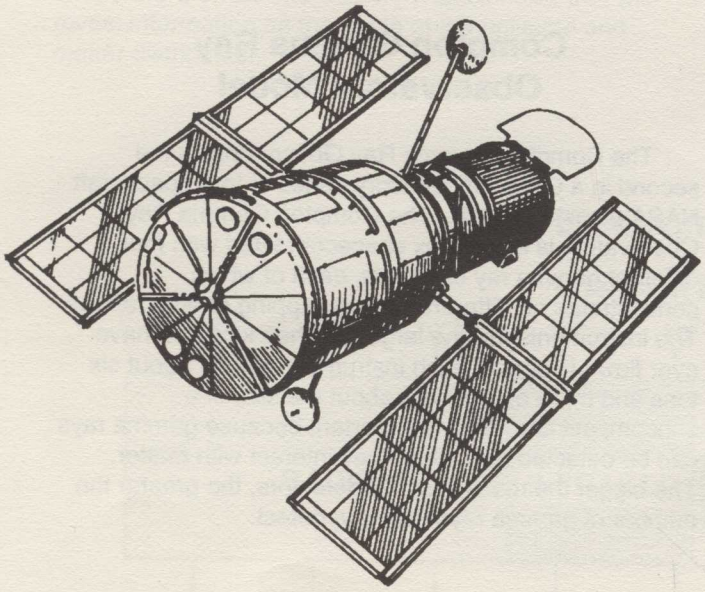
DEC 18 1996

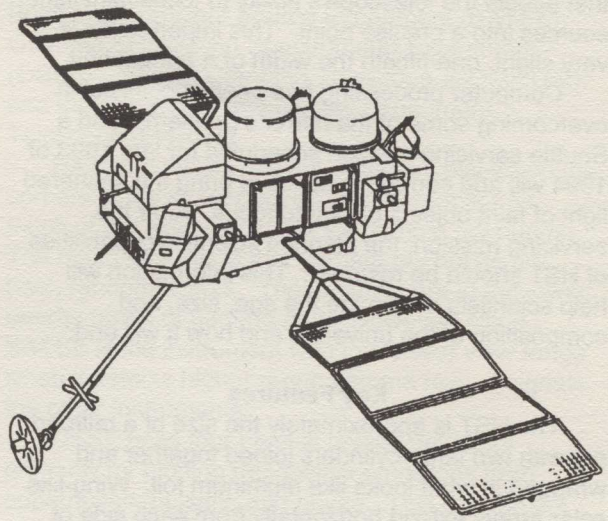
NASA'S Great Observatories

Why are space observatories important? The answer goes beyond twinkling stars in the night sky. Pockets of cold and hot air in Earth's atmosphere act as a hazy veil that many visible rays of light cannot penetrate. The atmosphere absorbs the majority of radiation from celestial bodies and distorts the types of light that do reach Earth's surface. Some types of radiation (like gamma rays) seldom reach Earth's surface. The radiation that Earth's atmosphere absorbs and distorts limits scientists' observations of stars, galaxies, and other celestial bodies. Even the most powerful ground based observatories can collect only a limited amount of data, but observatories in space collect data free from the distortion of Earth's atmosphere.

Space observatories contain advanced, highly-sensitive instruments such as telescopes (Hubble Space Telescope) and detectors (Compton Gamma Ray Observatory) that allow scientists to study radiation from neighboring planets and galaxies billions of light years away. By analyzing the spectrum of radiation emitted or absorbed by an object, scientists can determine temperature, chemical composition, and motion of an object. The light emanating from these far off celestial bodies takes billions of years to reach the observatories, so scientists can actually look into the past and learn what was happening in the universe when the universe was young. The data that these observatories gather help scientists understand star and galaxy formation, and the origination and evolution of our universe.

Hubble Space Telescope





Compton Gamma Ray Observatory

NASA, in conjunction with other countries' space agencies, commercial companies, and the international scientific community, has built observatories like the Hubble Space Telescope and the Compton Gamma Ray Observatory to find the answers to these and other questions.

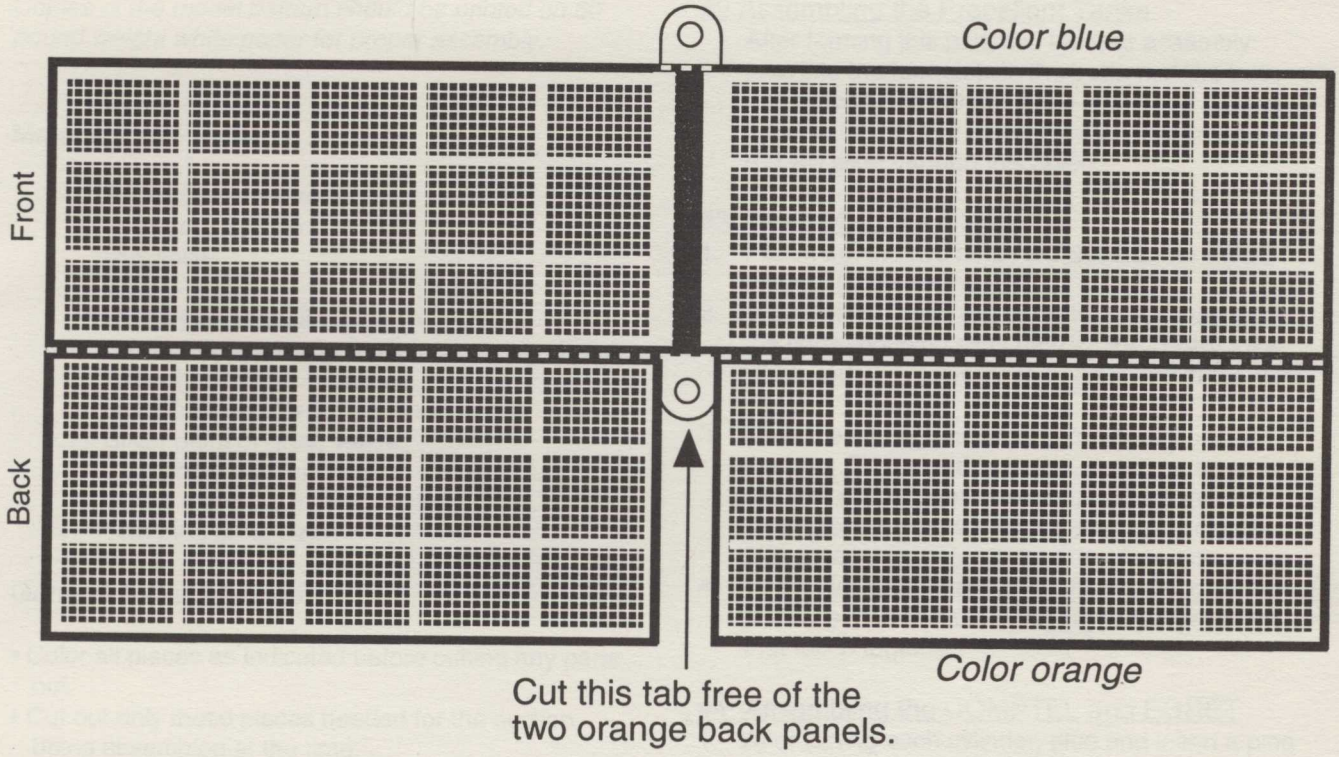
Hubble Space Telescope

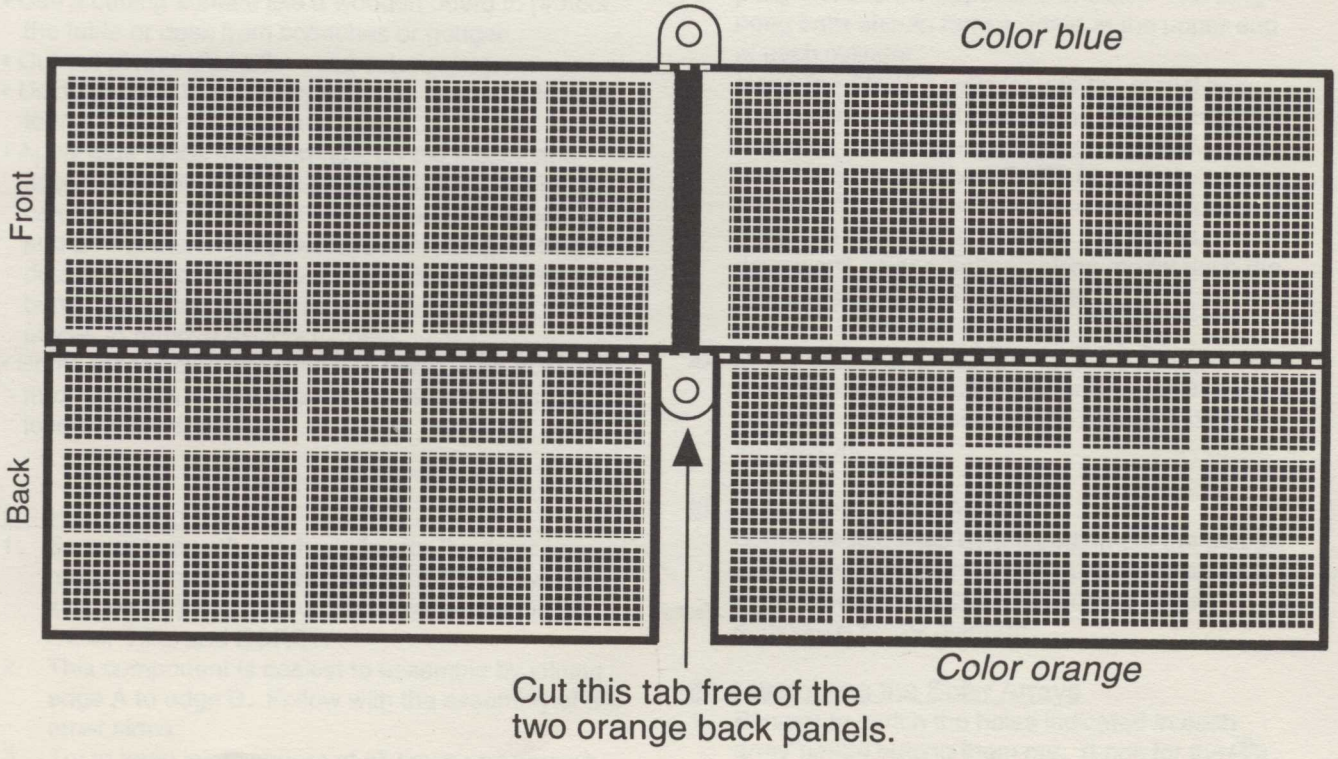
NASA's Hubble Space Telescope (HST) was deployed from Space Shuttle *Discovery* into Earth orbit in April of 1990. It is a product of two decades of research and development by 10,000 scientists and engineers at various NASA centers, private companies, universities, and the European Space Agency. The HST, the most complex and sensitive optical telescope ever made, was built to study the cosmos from low-Earth orbit for 10 to 15 years or more.

Scientific Objectives of HST

Scientists designed the HST to provide fine detail imaging, produce ultraviolet images and spectra, and detect very faint objects. Today, the HST is meeting the first two of these three objectives. A planned servicing mission will restore HST's ability to study very faint objects. HST has provided spectacular images and data of a giant storm on Saturn, unexpected clouds of hydrogen gas near our Galaxy, and the birth and rebirth of stars in older star clusters.

Solar Arrays





O'REILLY®

ST



Designing Social Interfaces

PRINCIPLES, PATTERNS, AND PRACTICES FOR IMPROVING THE USER EXPERIENCE

Christian Crumlish
& Erin Malone

[4]

Where's the Rest of Me?

We all perform. It's what we do for each other all the time, deliberately or unintentionally. It's a way of telling about ourselves in the hope of being recognized as what we'd like to be.

RICHARD AVEDON

THE CORE AROUND WHICH social sites revolve is people—who they are, how you know them, what they are contributing. People and their presentations of self and their contributions make for a rich and intertwined community. Without understanding who you are, your friends won't know you. Potential connections won't trust what you say or be encouraged to connect to you. You won't be able to recognize your friends in the crowd of participants, and you won't necessarily trust new people if you don't have some way of formulating a picture of who they are.

The work that goes into defining themselves as well as building connections is often a significant hurdle for users who want to try a new site. Thus, having a well-established identity and network on one site is a considerable deterrent for switching to other sites. Help your users by automating as much as possible; aggregating activity such as comments or reviews and presenting user status streams of connections creates a rich and interesting default without requiring much work from your users.

There is a growing belief that a person's identity belongs to him and not to the software or service in which the data has been created. The proliferation of socially enabled sites that do not interoperate means

[6]

Would You Buy a Used Car from This Person?

What is reputation? It's the general opinion (judgment) (more technically, a social evaluation) of (and by) the public (or a group or a person) toward an entity (person, a group of people, or an organization or brand or object)—as distinct and different from the background (others)—concerning the likelihood of the entity to behave in a certain way in the future [under certain circumstances]. It is a ubiquitous, spontaneous and highly efficient mechanism of social control.

TED NADEAU, REPUTATION 2.0 (*http://www.slideshare.net/ted.nadeau/sxsw-2007-reputation-20*)

Remember how you felt before the first day at a new school? Maybe it was frightening, the fear of the unknown, a new social environment with invisible rules and, for better or worse, a clean slate. Maybe it was exciting, a chance to reinvent yourself. Until it turns out your enemy from summer camp arrived just before you, talking behind your back and tainting your reputation before you even had a chance to make your first impression.

What? This has never happened to you? Nevermind... maybe I'm flashing back to high school.

Reputation online is another one of those social problem spaces that models the real world, connects to the real world, and in some ways simply extends the real world. One's reputation online might not be

precisely the same thing as one's "real world" reputation, but the principles are similar. Reputation is largely a matter of context. Are you known to be a certain way in your family, another among your fellow congregants, a third on the dance floor, a fourth on the mailing list, and yet another to total strangers via Google?

Including reputation metrics and services in your social interface is somewhat less ambitious than trying to measure people's real-world reputations or even trying to capture their online, virtual reputations. You can simply focus on the communities you are fostering in your application, the values you are trying to instill in the environment, the behaviors you wish to encourage, and the types of people you want to engage ever more deeply in your social environment.

Still, this is no small task, and a large part of it is determining the sorts of reputation patterns that you want to apply, because they all come with consequences and side effects. (Implementing a reputation platform is also an ambitious engineering task, beyond the scope of this book, but ably described in *Building Web Reputation Systems* by F. Randall Farmer and Bryce Glass [O'Reilly].)

Reputation Influences Behavior

A person participating in a social structure expects to develop a reputation and hopes for insight into the reputations of others, but each designed model of participation and reputation embodies its own set of biases and incentive structures. Balancing these forces determines in large measure the success or failure of a social system.

Note that we are talking here about "people reputation" and not "object reputation." We're talking "she's really helpful," not "huge appetizers but too much salt."

There is a virtuous cycle of activity—contribution and consumption—and each action feeds the reputation engine. As users contribute and are active on your service—sharing, writing reviews, collecting, publishing and so on, they are creating content for others in the community to consume and rank through reading, voting, and rating. This gives information about quality to the site or application which in turn rewards and incentivizes these contributors to keep contributing.

Fundamentally, a reputation system involves tracking desirable behavior and then recognizing it publicly. So, any well-designed reputation system is going to start with an inventory of desirable behaviors. Do

you want to ensure that people try out a certain feature, strive for higher-quality contributions, or log endless hours responding to others? Just as managers say, "You can't manage what you can't measure," the same applies to reputation. You can't acknowledge what you aren't tracking.

When defining desirable behaviors, be sure to think about the individuals, the groups, and the site itself. The pact you make with users to feed the reputation system involves a tradeoff: they reward good content, they share opinions with their network, they downgrade bad content, they become subject matter experts, and they share their tastes with an expectation that the site will adapt. This behavior makes it possible for you to adapt and personalize as they become more engaged; it gives you the ability to feature high-rated items creating experts, and power a recommendations algorithm.

Note that a functioning reputation system requires stable, persistent identities (as discussed in Chapter 3).

Competitive Spectrum

The first decision you need to make about developing your reputation system is to determine how competitive or cooperative a culture you would like to cultivate on your site. This determination will help you to select the appropriate reputation patterns to apply. What kind of community are you building? Tough or nurturing? What role do you want reputation to play there? Hard-edged and unforgiving, gentle and caring, or somewhere in between?

What

When a new or existing community requires a reputation system, the designer must pay careful attention to the degree of *competitiveness* the community ought to exhibit. Haphazardly introducing competitive incentives into noncompetitive contexts can create problems and might cause a schism within the community.

Use when

Use this pattern when choosing the type of reputation system to design for a community.

How

The chart in Figure 6-1 attempts to describe a community in terms of its "competitiveness." This is a broad term, but here it is used to describe a combination of things: the *individual goals* of community members, and to what degree those goals coexist peacefully or conflict; the *actions* that community members engage in, and to what degree those actions might impinge on the experiences of other community members; and to what degree *person-to-person* comparisons or contests are desired.

FIGURE 6-1. The Competitiveness Spectrum helps you choose reputation patterns appropriate to the tone and culture of your community.

CARING	COLLABORATIVE	CORDIAL	COMPETITIVE	COMBATIVE
GOALS				
Members are motivated by *helping* other members—giving advice, solace or comfort	Member goals are largely shared ones. Members work together to achieve those goals.	Members have their own intrinsic motivations, but these goals need not conflict with other members' goals.	Members share the same goals, but must compete against one another to achieve them.	Members share opposing goals: for one member to achieve these goals, others must necessarily be *denied* their own.
USE REPUTATION TO...				
Identify senior members of good standing so that others can find them for advice and guidance.	Identify community members with a proven track record of being trustworthy partners.	Show a member's history of *participation*, that others might get a general sense for their interests, identity, and values.	Show a member's level of *accomplishment*, that others might acknowledge (and admire) their level of performance.	Show a member's history of accomplishments, including other members' victories and defeats against them. Reputation is used to establish bragging rights.
REPRESENT REPUTATION WITH...				
Accept volunteers (of good standing) from the community to wear an *Identifying Label*: "Helpful" or "Forum Leader." New members can trust these folks to help initiate them into the community.	Use *Named Levels* to communicate members' history and standing; members with higher ranks should be trusted more easily than newbies.	Consider Statistical Evidence to highlight a member's contributions. Just show the facts and let the community decide their worth. Optionally, *Top X* designations can highlight members with numerous valued contributions.	Allow easy comparisons between members with *Numbered Levels*. Provide mini-motivations by awarding *Collectible Achievements*.	Let a member track her own progress by assigning *Point Values* to different actions. *Rank* members against one another, displaying winners and losers.

This "Competitiveness Spectrum" is admittedly subjective, and it would not be surprising to find many examples for which this model does not hold up exactly as illustrated. The trick is to have *any* kind of framework to start from, not to have a definitive and comprehensive model.

Depending on the relative level of competitiveness present in your community, we'll recommend appropriate reputation patterns.

Why

- The proper level of competitiveness, or lack thereof, can encourage and incite your users do more of what you want (contribute and participate) rather than just passively consume.

Levels

Clearly defined levels of accomplishment provide an explicit measurement of reputation, capturing and sharing an empirical assessment of the individual's reputation in the social structure. Levels are typically named or numbered. Named levels generally have an explicit rank order, such as shown in Figure 6-2, so in a sense both variations can be viewed as numbered levels.

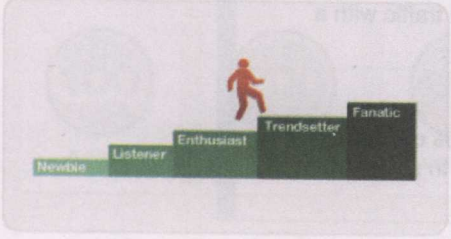


FIGURE 6-2
One form of reputation is a ranking system that characterizes your level of experience or accomplishment.

Named Levels

What

Participants in a community need some way to gauge their own personal development within that community: how far they've progressed, or how deeply they've interacted with the community or its offerings. Additionally, these same measures can be used to compare members, to understand who has more or less experience in the community (as long as a high degree of comparability is not desired), which, as Figure 6-3 demonstrates, is exactly what traffic and navigation app Waze does.

BrodartConvert
BRO DART

LOS ANGELES PUBLIC LIBRARY
3 7244 2231 3902 4

O'REILLY®

Designing Social Interfaces

Designers, developers, and entrepreneurs today must grapple with creating social interfaces to foster user interaction and community, but grasping the nuances and the building blocks of the digital social experience is much harder than it appears. Now you have help.

In the second edition of this practical guide, UX design experts Christian Crumlish and Erin Malone share hard-won insights into what works, what doesn't, and why. With more than 100 patterns, design principles, and best practices, you'll learn how to balance opposing forces and grow healthy online communities by co-creating the experience with your users.

- Understand the overarching principles before applying tactical design patterns
- Cultivate healthy participation and rein in misbehaving users
- Learn patterns for adding social components to an existing site
- Encourage users to interact with one another, whether it's one-to-one or many-to-many
- Use a rating system to build a social experience around products or services
- Orchestrate collaborative groups and discover the real power of social networks
- Explore numerous examples of each pattern, with an emphasis on mobile apps
- Learn how to apply social design patterns to enterprise environments

"*Designing Social Interfaces* is a fantastic reference for people who want to find the right design patterns fast. If you need to understand how to connect your users to your product and to each other, this book is perfect for you."

—Laura Klein
author of *UX for Lean Startups*

"Chock-full of visual examples, *Designing Social Interfaces* is a smart and readable reference that will help you apply appropriate design patterns so you don't waste time re-inventing the wheel."

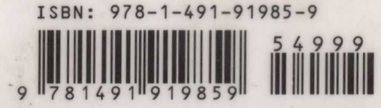
—Kate Rutter
UX Instructor and Visual Thinking Consultant, Intelleto

Christian Crumlish is VP of Product at 7 Cups of Tea (7cups.com). He also served as director of product at CloudOn, director of messaging products at AIM, and curator of the Yahoo! Design Pattern Library.

Erin Malone, Principal with Tangible UX, has over 20 years of experience leading design teams and developing social experiences for a host of startups and large enterprises. She's the former director of the team that produced the Yahoo! Design Pattern Library.

INTERNET APPLICATIONS / SOCIAL WEB

US $49.99 CAN $57.99
ISBN: 978-1-491-91985-9





Twitter: @oreillymedia
facebook.com/oreilly



SECOND EDITION

Designing Social Interfaces

Crumlish & Malone

510.78W
C956
2015

ST

O'REILLY®



Designing Social Interfaces

PRINCIPLES, PATTERNS, AND PRACTICES FOR IMPROVING THE USER EXPERIENCE

Christian Crumlish & Erin Malone



FIGURE 6-3
Waze mobile GPS application uses crowdsourced information to let drivers know about accidents or police on the road. Participation is rewarded with a variety of named levels based on points accrued (Waze iOS application).

Use when

You want consumers to be able to discover and identify high-quality contributors.

The community is competitive, but not highly competitive. Although named levels can have a competitive edge to them (my Wookie beats your Jawa!), they are perceived as less competitive than some other patterns (e.g., "Rankings," "Points," and "Numbered Levels"), perhaps because they are less empirical in nature.

You want your users to be able to track their individual growth in the community, and suggest ways that they can attain the next level in the hierarchy.

How

Define a family of reputations on a progressive continuum. Each level that is achieved is higher than the one before it. Levels are given unique names, which can give them a fun and approachable quality. Quick comparisons between levels, however, become slightly more difficult (see Figure 6-4).



FIGURE 6-4
Levels on Untappd are based on quantity of beer consumed. The amounts required for each level are not actually exposed until the user reaches that level (Untappd iOS application).

[13]

Heard It Through the Grapevine

Some commons-based peer production efforts are less self-conscious on the part of the users, and emerge more as a function of distributed coordinate behavior, like del.icio.us or Flickr. The critical defining feature of these "enterprises" is that they rely primarily on social information flows, motivations, and relations to organize the group. Individuals self-identify, mostly, for tasks, and through a variety of peer-review mechanisms contributions get recognized by the group and incorporated into what emerges as the collaborative output.

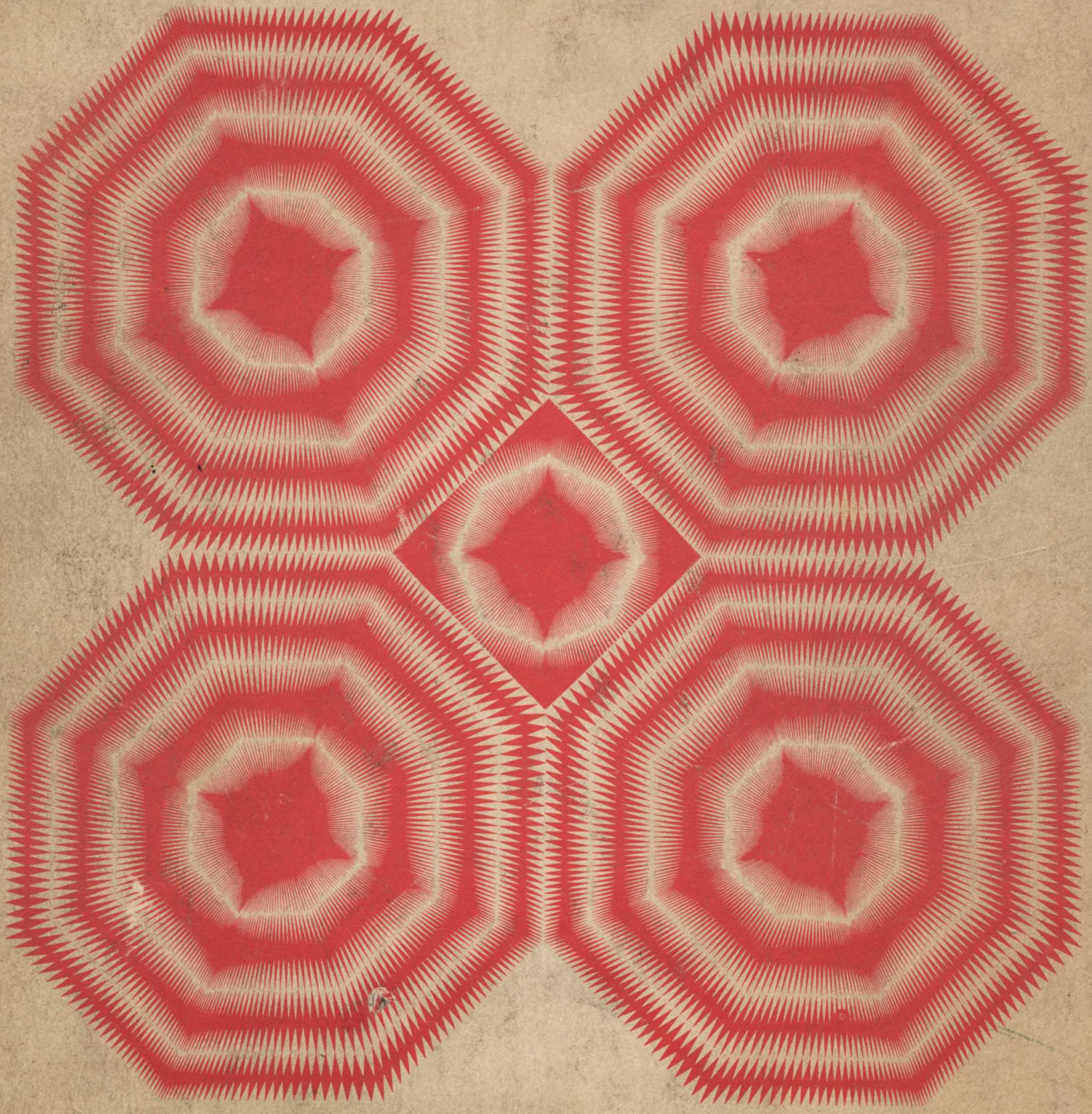
YOCHAI BENKLER INTERVIEW IN *OPENBUSINESS* (ABOUT HIS BOOK *THE WEALTH OF NETWORKS*)

Keeping Up

As with "Web 2.0" and "synergy," the buzzphrase "social media" has taken on a life of its own and has already spawned its own mutant bastard: "social media marketing." Similar to those other terms—and much Internet jargon—"social media" means different things to different people. As noted at the beginning of this book, we prefer to use the term to refer to social production and consumption of media objects, not merely as a lazy synonym for social networking in general. This chapter is devoted to the collection, sharing, and creation of social objects. As Hugh MacLeod put it in a post on his weblog, called "Social Objects for Beginners" (*http://www.gapingvoid.com/Moveable_Type/archives/004390.html*):

LASERS

Selected Reprints



published by
American Association of Physics Teachers

Teptmount
BMC

LASERS:
Selected Reprints

edited by Donald C. O'Shea
Georgia Institute of Technology
Atlanta, Georgia

and

Donald C. Peckham
St. Lawrence University
Canton, New York

535.58 L3435
001

APR 14 1984

published by
American Association of Physics Teachers

June 1982

MAR 02 1984